Process and Context Switch

Part I

The Kernel Abstraction

2569/2026

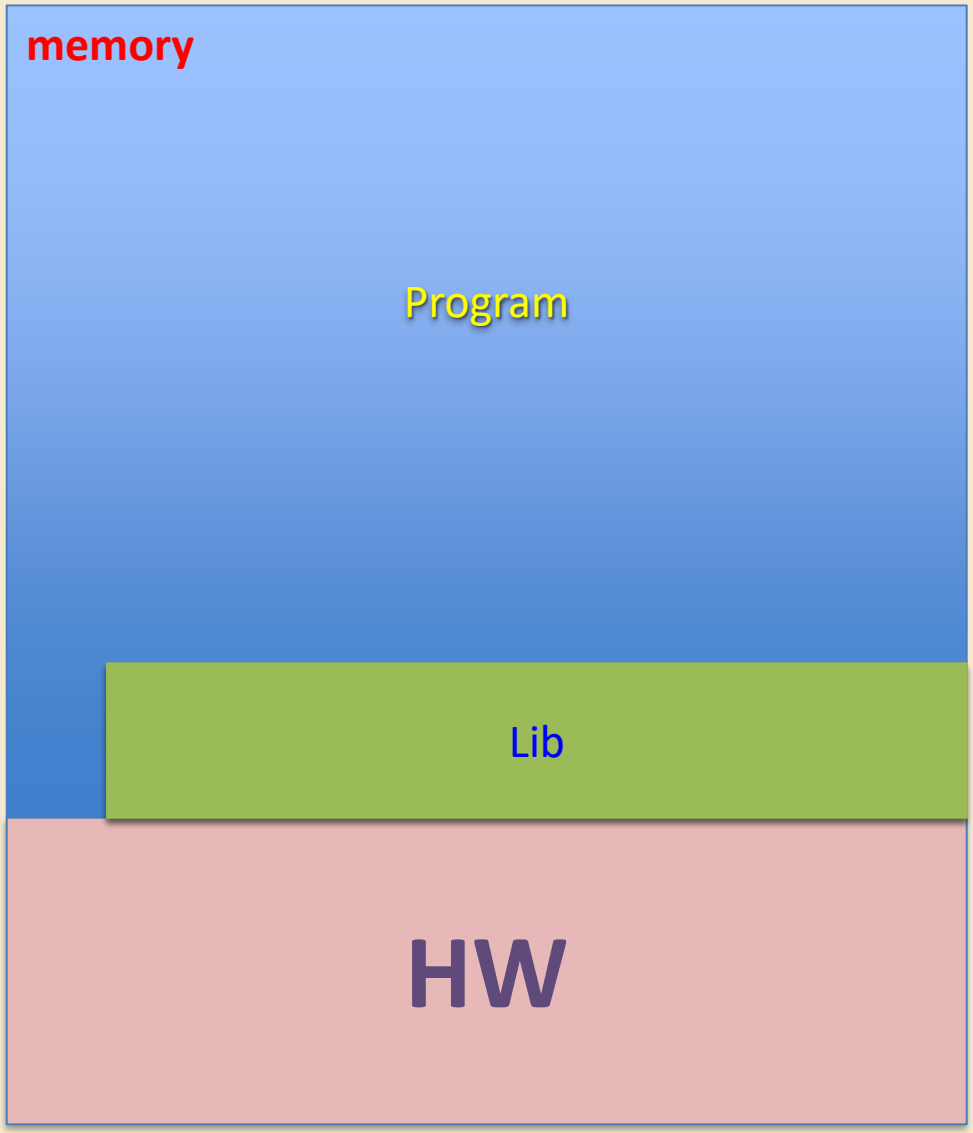
Slides adopted from CSE 451 class at UW, CS162 class at Berkeley and CSE 421/521 class at UB

Computer Hardware (PC)

- Parts variety
  - Different manufacturers
    - Different instruction sets
  - Different versions of hardware

Question #1

- How to deal with the variety of hardware in your opinion?


memory
Program
Lib
HW

Tasks

- Past...
  - Single-task system
    - 1 task at a time
    - Dedicating all resources (HW) to 1 task
- Today...
  - Multiuser/Multitasking system
    - More than 1 task are running
    - Resources are shared between tasks
    - More than 1 user can access to the system
    - Users can keep their resources to themselves or share them with the others

Question #2

- By transitioning from a single-task system to a multitasking system, what adjustments do we need to make to the system?

The model will become

What does an OS do...

One of the major goals of OS is...

Question #3: Protection: WHY?

By applying protection concepts to processes and the kernel, what could potentially impact the system?

Question #4: Protection: How? (HW/SW)

How can we implement the protection that was mentioned earlier?

Hardware Protection

- CPU instructions are divided into
  - Privileged instructions
  - Non-privileged instructions

Privileged instructions

- Examples?

Dual-mode operation

- At first, it was done by software

Hardware Support: Dual-Mode Operation

- Kernel mode
  - Execution with the full privileges of the hardware
  - Read/write to any memory, access any I/O device, read/write any disk sector, send/read any packet
- User mode
  - Limited privileges
  - Only those granted by the operating system kernel
- On the x86, mode stored in **EFLAGS** register
- On the MIPS, mode in the status register

Hardware Support: Dual-Mode Operation

- Privileged instructions
  - Available to kernel
  - Not available to user code
- Limits on memory accesses
  - To prevent user code from overwriting the kernel
- Timer
  - To regain control from a user program in a loop
- Safe way to switch from user mode to kernel mode, and vice versa

User Mode

- Application program
  - Running in process

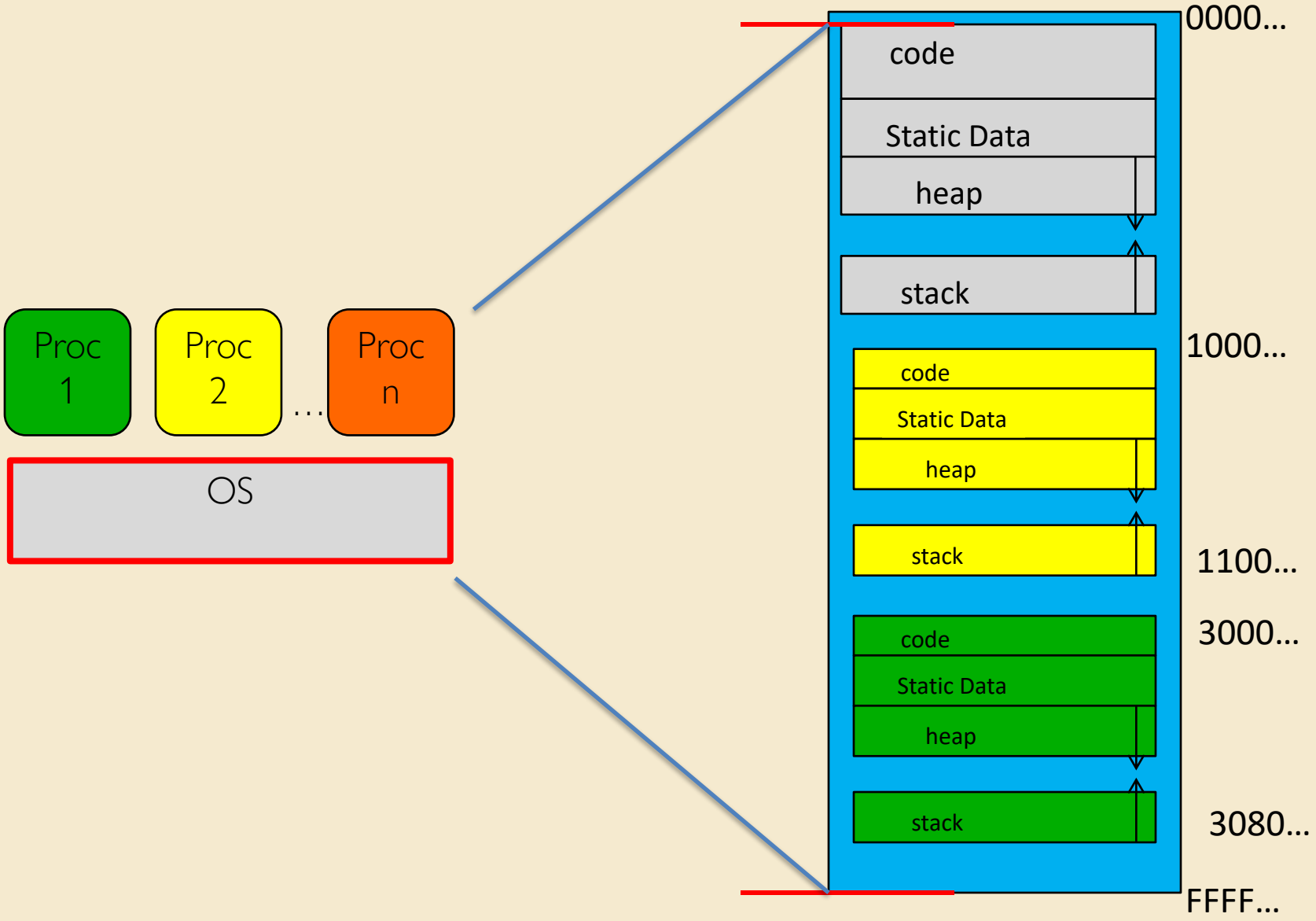
Virtual Machine:VM

- Software emulation of an abstract machine
  - Give programs illusion they own the machine
  - Make it look like HW has feature you want
- 2 types of VM
  - Process VM
    - Supports the execution of a single program (one of the basic function of the OS)
  - System VM
    - Supports the execution of an entire OS and its applications

Process VMs

- GOAL:
  - Provide an isolation to a program
  - Portability (Program)

Kernel mode & User mode

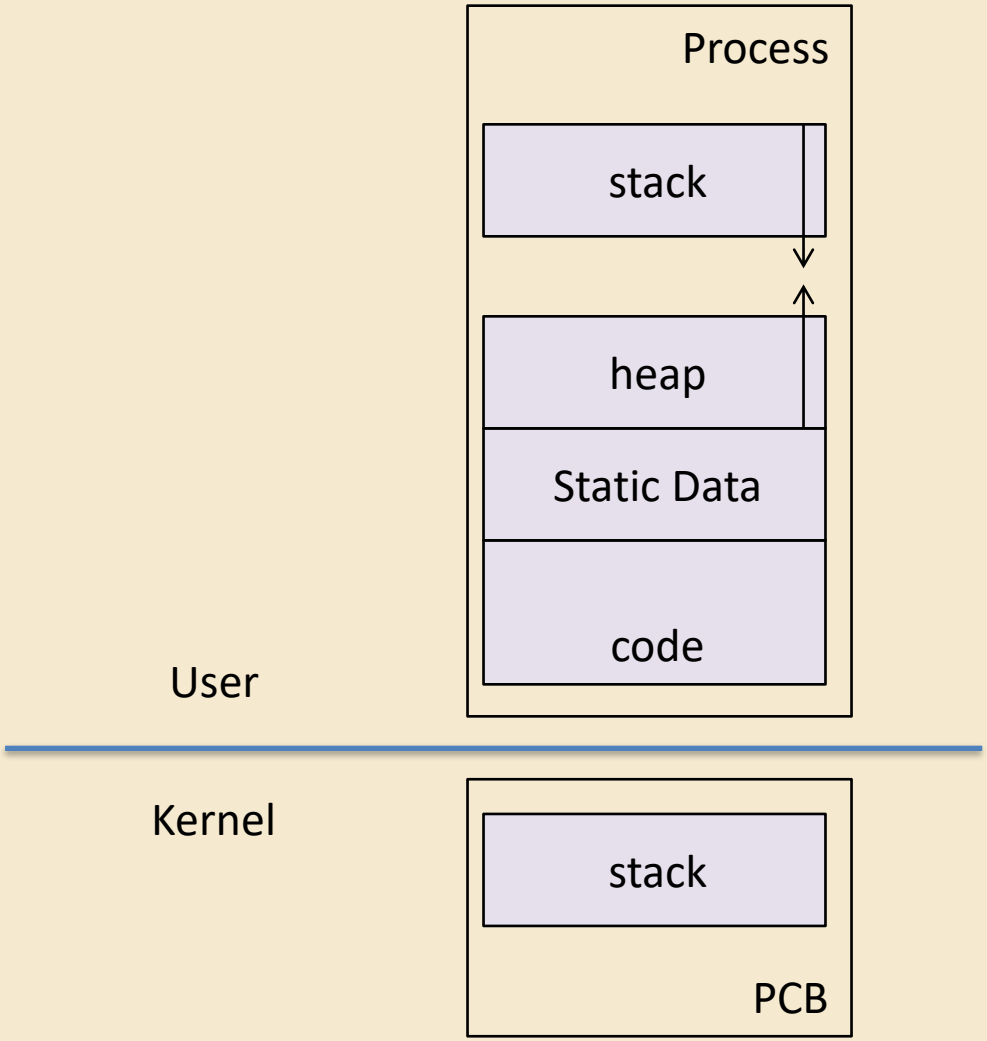


Process Abstraction

- Process: an *instance* of a program, running with limited rights
  - Thread: a sequence of instructions within a process
    - Potentially many threads per process (for now 1:1)
  - Address space: set of rights of a process
    - Memory that the process can access
    - Other permissions the process has (e.g., which system calls it can make, what files it can access)

Process

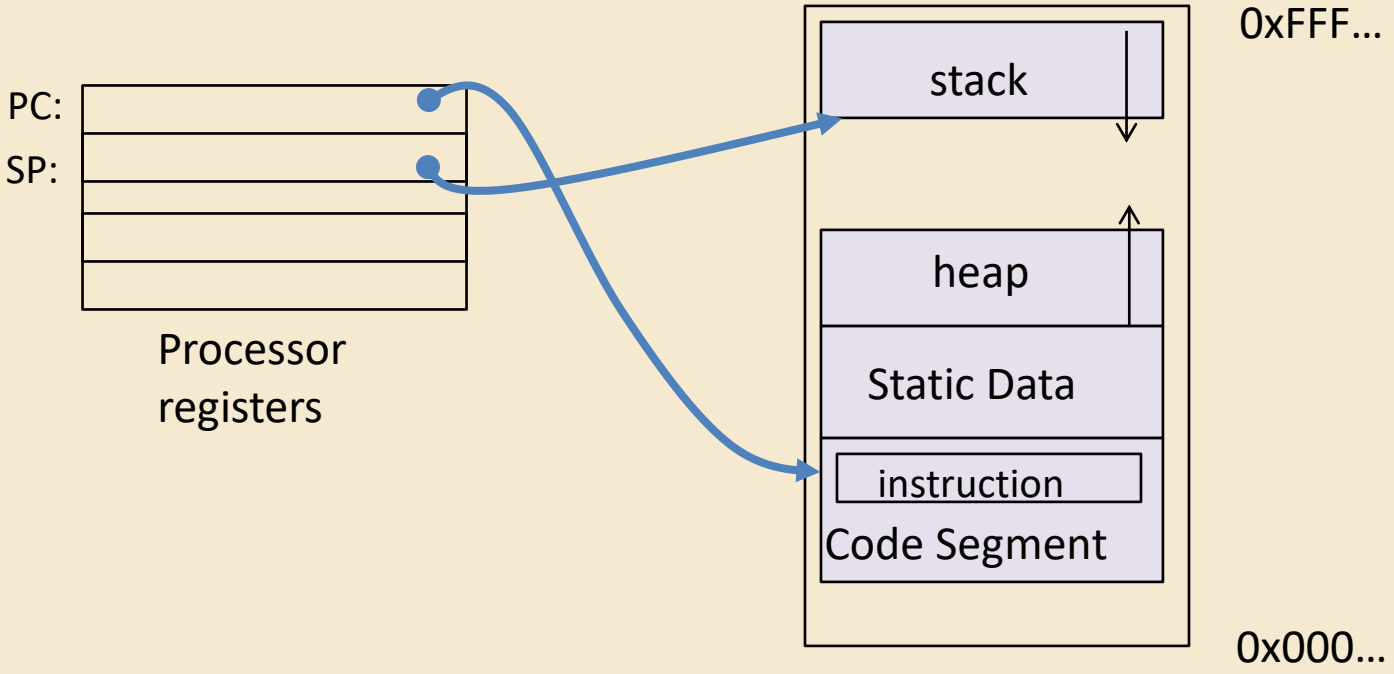
- 2 parts
  - PCB in kernel
  - Others in user



Process Control Block: PCB

- Kernel represents each process as a process control block (PCB)
  - Status (running, ready, blocked, ...)
  - Registers, SP, ... (when not running)
  - Process ID (PID), User, Executable, Priority, ...
  - Execution time, ...
  - Memory space, translation tables, ...
- Kernel Scheduler maintains a data structure containing the PCBs
- Scheduling algorithm selects the next one to run

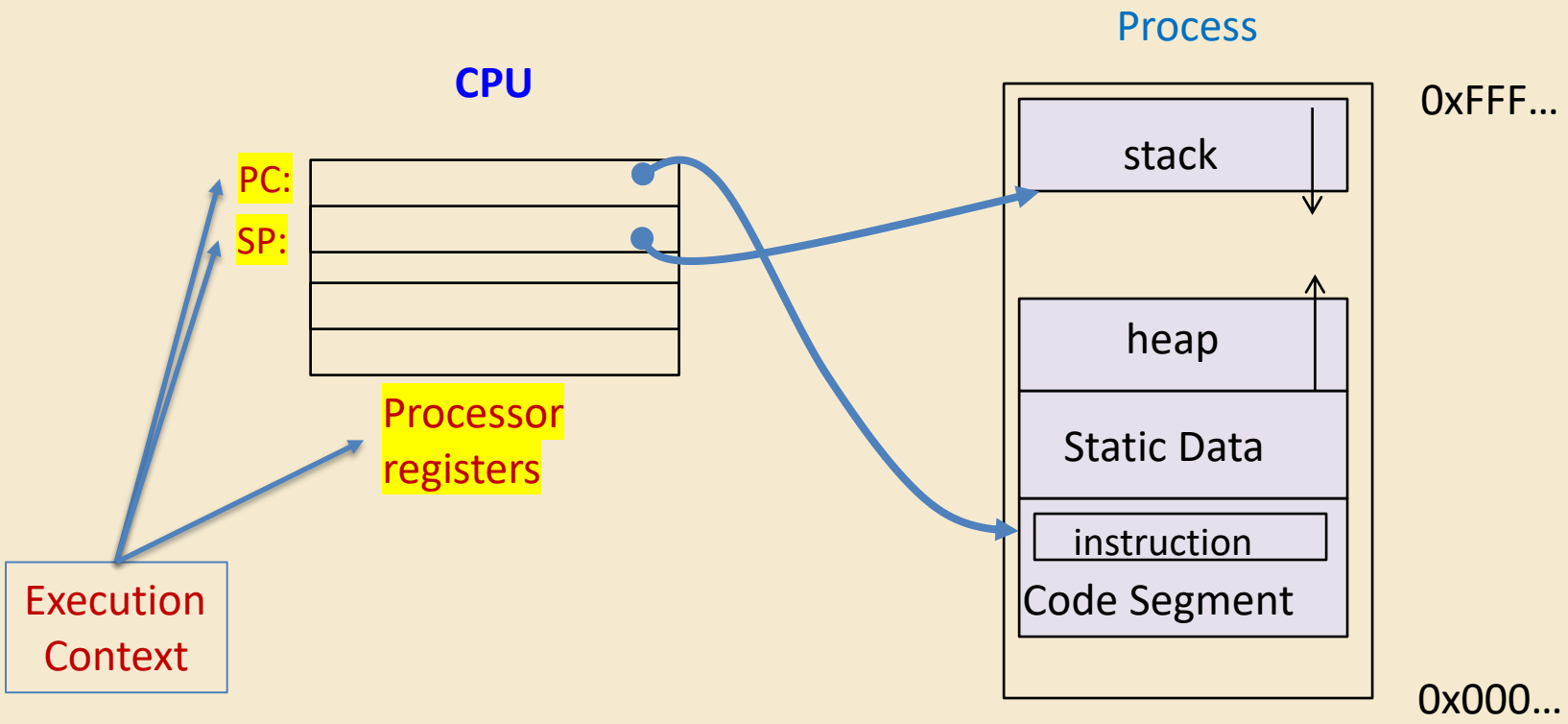
Address Space: In a Picture



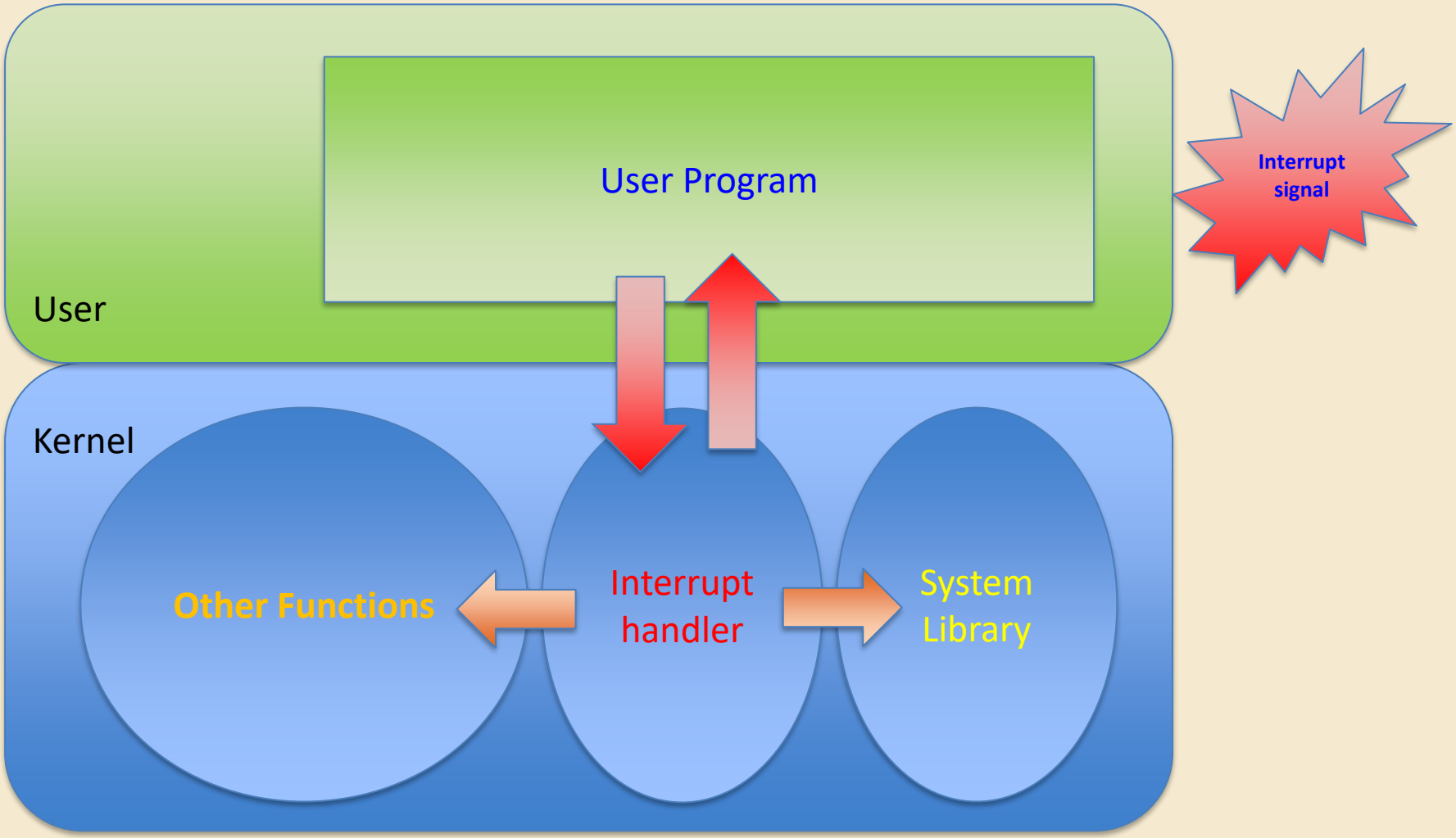
At this points, we should know...

- Process concept
  - A process is the OS abstraction for executing a program with limited privileges
- Dual-mode operation: user vs. kernel
  - Kernel-mode: execute with complete privileges
  - User-mode: execute with fewer privileges
- Safe control transfer
  - How do we switch from one mode to the other?

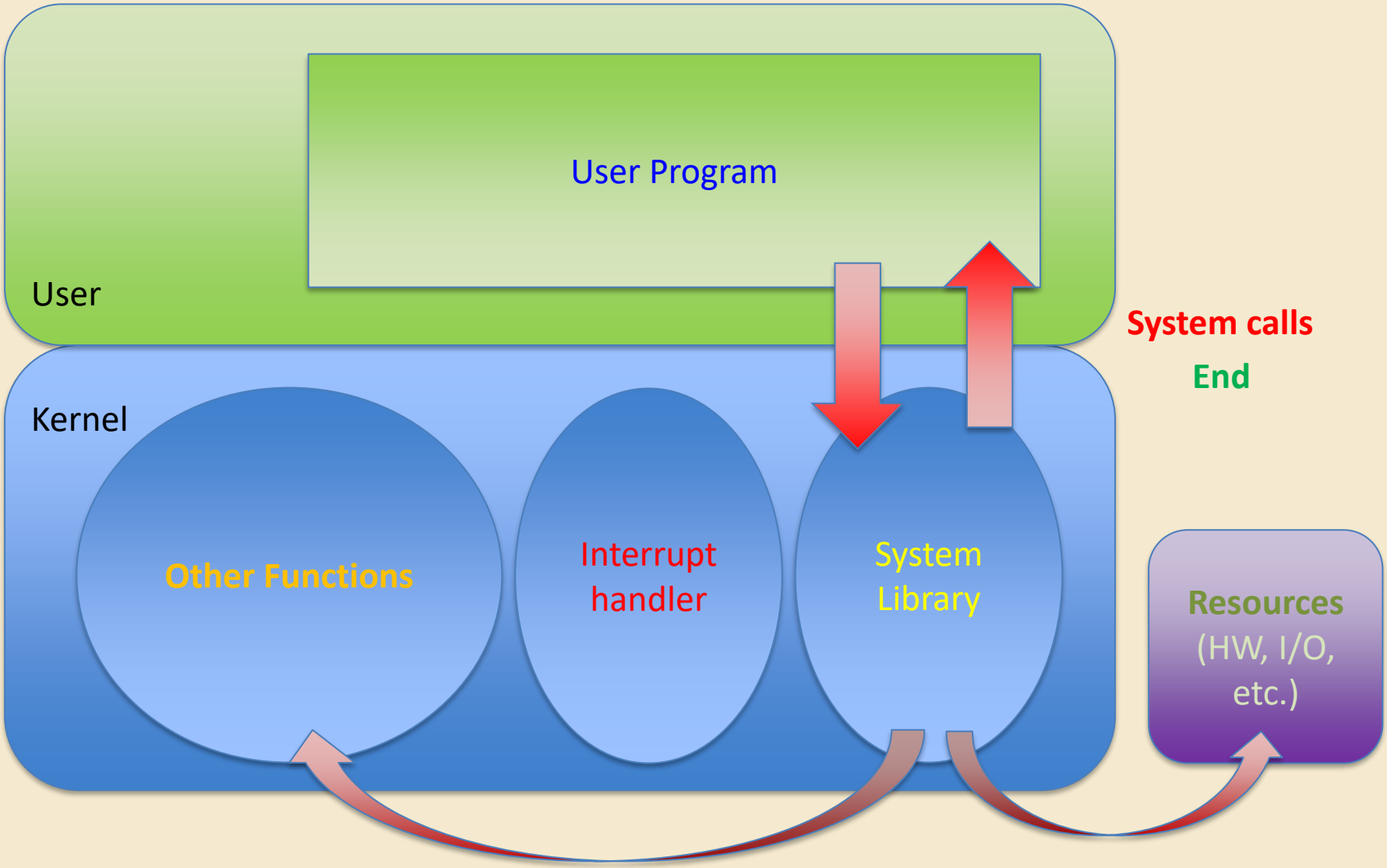
Process and Execution context



Interrupt


User Program
User
Interrupt signal
Kernel
Other Functions
Interrupt handler
System Library

System calls



Mode Switch

- From user mode to kernel mode
  - Interrupts
    - Triggered by timer and I/O devices
  - Exceptions
    - Triggered by unexpected program behavior
    - Or malicious behavior!
  - System calls (aka protected procedure call)
    - Request by program for kernel to do some operation on its behalf
    - Only limited # of very carefully coded entry points

Mode Switch

- From kernel mode to user mode
  - New process/new thread start
    - Jump to first instruction in program/thread
  - Return from interrupt, exception, system call
    - Resume suspended execution
  - Process/thread context switch
    - Resume some other process
  - User-level upcall (UNIX signal)
    - Asynchronous notification to user program

Question #5

- In your opinion, how should mode switching be conducted to ensure data security and system stability?

Implementing Safe Kernel Mode Transfers

End of Part-I